

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 01

Team No.: 5

Project Title: OS Resource Management Simulator

Team Members

Roll Number	Student Name	Signature
2520030595	S.SRI SAI PRADYUMN	
2520030579	G.JASHWANTH	

1. Abstract

The OS Resource Management Simulator is a Linux-based educational application that demonstrates how an operating system manages and allocates major system resources. The simulator covers CPU scheduling, memory allocation, process management, and resource allocation with deadlock handling. For CPU management, it implements First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. For memory management, it demonstrates common allocation approaches such as First Fit, Best Fit, and Worst Fit. The simulator also models process states and resource requests and uses Banker's Algorithm to identify safe and unsafe resource-allocation states. It calculates relevant performance and allocation metrics and provides visual representations such as Gantt Charts and allocation tables. The project applies core Operating Systems concepts including process management, CPU scheduling, memory management, resource allocation, and deadlock avoidance. The proposed solution will be implemented in Linux using C/C++, providing a practical environment for understanding how operating systems manage limited resources.

2. Problem Statement

An operating system must efficiently manage limited resources such as CPU time, memory, and system resources while allowing multiple processes to execute safely and efficiently. Different scheduling and allocation strategies can produce different execution orders, waiting times, memory utilization, and resource-allocation outcomes. Deadlocks can also occur when processes compete for resources in an unsafe manner. Understanding these interactions manually can be difficult, especially as the number of processes and resources increases. This project addresses the problem by developing a simulator that models major OS resource-management activities, applies scheduling and memory-allocation algorithms, checks resource-allocation safety using Banker's Algorithm, and displays the results in an easy-to-understand form. The project is directly relevant to Operating Systems because resource management is a fundamental function required for efficient and safe system operation.

3. Objectives

- Implement CPU scheduling algorithms including FCFS, SJF, Priority Scheduling, and Round Robin.

- Implement memory allocation strategies such as First Fit, Best Fit, and Worst Fit.
- Simulate process states and maintain processes according to their resource and CPU requirements.
- Implement resource allocation and Banker's Algorithm for deadlock avoidance and safe-state checking.
- Calculate relevant scheduling, memory-utilization, and resource-allocation performance metrics.
- Provide Gantt Charts and allocation tables to visualize resource-management decisions.
- Compare the behavior and performance of different resource-management strategies.

4. Proposed Methodology

The project will be developed and executed in a Linux/Ubuntu environment using C or C++. The user first enters process details such as process ID, arrival time, burst time, priority, memory requirement, and resource requirements. The simulator validates the input and passes the process information to the selected resource-management module. The CPU scheduling module applies the selected scheduling algorithm and calculates completion, waiting, turnaround, and response times. The memory-management module applies First Fit, Best Fit, or Worst Fit and reports allocation and utilization. The resource-allocation module maintains available, allocated, and required resources and uses Banker's Algorithm to determine whether a requested allocation leaves the system in a safe state. The simulator then displays Gantt Charts, memory-allocation results, safe/unsafe status, and performance values. A comparison module can be used to compare the results produced by different algorithms. The implementation will use standard Linux development tools and POSIX-compatible programming practices where required.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API	Purpose in the Project
CPU Scheduling	Schedules processes for CPU execution using different scheduling algorithms.
FCFS, SJF, Priority Scheduling	Determines process execution order and calculates scheduling metrics.
Round Robin	Provides time-sliced CPU scheduling using a configurable time quantum.
Memory Management	Allocates available memory to processes using First Fit, Best Fit, and Worst Fit.
Process Management / Ready Queue	Maintains process states and processes waiting for CPU or resources.
Resource Allocation & Banker's Algorithm	Checks resource requests and determines safe or unsafe allocation states.
POSIX/Linux Process Mechanisms	Supports the Linux-based implementation and process-related programming where required.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030595	S.SRI SAI PRADYUMN	Project planning, requirements, documentation, and CPU scheduling implementation
2520030579	G.JASHWANTH	Memory/resource allocation, Gantt Chart and performance module, testing, debugging, and GitHub documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment.
C / C++	Implementation of resource-management algorithms and simulator logic.
GCC / G++	Compilation of the C/C++ source code.
VS Code / Code Editor	Writing, debugging, and maintaining project source code.
Git / GitHub	Version control and project repository management.
POSIX/Linux APIs	Support for Linux-based process and system programming where required.

8. Expected Outcome

The completed system is expected to provide a functional Linux-based OS Resource Management Simulator. Users will be able to enter process, memory, and resource information, select appropriate management algorithms, and view the resulting execution and allocation decisions. The simulator will provide Gantt Charts for CPU scheduling, memory-allocation results, resource-allocation status, safe/unsafe-state information, and relevant performance metrics. The final outcome will demonstrate the practical behavior of CPU scheduling, memory management, process management, resource allocation, and deadlock avoidance, connecting theoretical Operating Systems concepts with an executable implementation.

9. GitHub Repository

GitHub Repository Link https://github.com/govindapuramjashwanth-cell/TechSphere_OSSP

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr. D. Bheekya

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____